

Programação PL/SQL

Para Banco de Dados

jmmaciel@upf.br

PL/SQL

- Procedural Language/Structured Query Language
- Principais características:
 - executar comandos SQL para manipular dados em tabelas
 - criação de variáveis e constantes
 - cursores para tratar resultados de consultas
 - criação de registros para guardar resultados de consultas
 - tratamento de erros
 - controle de fluxo (if-then-else, case) e repetição (loop, for, while)

PL/SQL

- Vantagens
 - Versatilidade
 - Portabilidade
 - Integração com o SGBD
 - Redução de tráfego de rede
- Ferramentas
 - SQL*PLus
 - SQL Developer
 - Oracle Forms

Estrutura de um bloco PL/SQL

DECLARE

-- Inicializações, declaração de constantes, variáveis e cursores

BEGIN

-- Comandos SQL, estruturas de programação e outros blocos PL/SQL

BEGIN

...

END;

EXCEPTION (opcional)

-- Tratamento de exceções, emissão de mensagens

END;

Declare

- Constantes, variáveis, cursores, estruturas e tabelas
- Tipos de dados primitivos (próximo slide)
- Tipos de dados compostos
 - RECORD
 - TABLE
- Tipos referenciados
 - REF CURSOR

Tipos de dados

TIPOS	DESCRIÇÃO
CHAR	Alfanumérico, tamanho fixo, limite: 2000 caracteres
CHARACTER	Idêntico ao CHAR, mantém compatibilidade com outras versões SQL
VARCHAR2	Alfanumérico, tamanho variável, limite: 4000 caracteres
VARCHAR E STRING	Idêntico ao VARCHAR2, mantém compatibilidade com outras versões SQL
CLOB (Character Long Object)	Alfanumérico, tamanho variável, limite 4 Gb
LONG	Alfanumérico, limites: 2 GB, apenas um por tabela
ROWID	Armazena os valores dos ROWIDs das linhas das tabelas
BLOB (Binary Long Object)	Binário, tamanho variável, limite: 4 Gb
BFILE (Binary File)	Armazena uma referência a um arquivo externo (que deverá localizar-se na mesma máquina do banco de dados, não permite referência remota)
RAW	Hexadecimais, tamanho variável, limite: 2 Kb
LONG ROW	Hexadecimais, tamanho variável, limite: 2 Gb
NUMBER	Numérico, limite: 38 dígitos Exemplo: NUMBER (10,2) armazena 10 números (8 inteiros e 2 decimais) SUBTIPOS: • DECIMAL • DEC • DOUBLEPRECISION • INTEGER • INT • NUMERIC • REAL • SMALLINT • FLOAT • PLS_INTEGER
BINARY_INTEGER	Numérico, positivos e negativos, limites: -2147483647 e 2147483647 SUBTIPOS: • NATURAL (limites: 0 e 2147483647) • NATURALN (limites: 0 e 2147483647, não aceita valores nulos) • POSITIVE (limites: 1 e 2147483647) • POSITIVEN (limites: 0 e 2147483647, não aceita valores nulos)
DATE	Data e hora (formato padrão: DD-MMM-YY)
TIMESTAMP	Data e hora (com milésimos de segundo)
BOOLEAN	Armazena os valores: TRUE, FALSE ou NULL

Declarações

- Constantes – a palavra **CONSTANT** deve aparecer antes do tipo de dado

```
DECLARE
    pi    CONSTANT NUMBER(9,7) := 3.1415927;
    ...
BEGIN ...
END; /
```

- Variáveis

```
v1    NUMBER(4) := 1;
v2    NUMBER(4) := DEFAULT 1;
v3    NUMBER(4) NOT NULL := 1;
```

- Escopo de variáveis
 - Variáveis definidas em um bloco são locais em um bloco e globais para os sub-blocos

Operadores

- **Atribuição** pode ocorrer de duas formas:

- operador :=

```
total := quant * valor;
```

- Utilizando select com a cláusula INTO

```
SELECT idaluno, nomealuno  
INTO    varid, varaluno  
FROM    aluno;
```

- **Concatenação**

- Operador || pode ser usado

```
SELECT 'Passo Fundo' || ' - ' || 'RS'  
FROM dual;
```

=> Obs: tabela dual é utilizada para retornar valores de expressões, mas que não extraem dados de tabelas de BD.

Herança de tipo e tamanho

- Constantes e variáveis podem herdar tipos de outras variáveis, de colunas e até mesmo de linhas inteiras de tabelas
- Permite acompanhar a evolução de esquemas, diminuindo a manutenção de blocos PL/SQL

Exemplos:

```
variavel_2 variavel_1%Type; -- herda tipo de outro variável  
variavel tabela.nomecoluna%Type; -- herda tipo de coluna  
variavel tabela%Rowtype; -- herda tipo de uma linha inteira
```

Expressões SQL em blocos PL/SQL

- Comandos DML (SELECT, INSERT, UPDATE e DELETE) podem ser utilizados dentro de um bloco PL/SQL.
- Um comando SELECT deverá receber obrigatoriamente a cláusula INTO para que o resultado seja armazenado em variáveis e deverá retornar apenas uma linha.
- Caso mais de uma linha seja retornada, apresentará o erro: `too_many_rows` e se não retornar nenhuma linha, apresentará o erro: `no_data_found`.
- O tratamento destas exceções é apresentado mais adiante

```
DECLARE
    V_RA ALUNO.RA%TYPE;
    V_NOME ALUNO.NOME%TYPE;
BEGIN
    SELECT RA, NOME
    INTO V_RA, V_NOME
    FROM ALUNO
    WHERE RA=1;
END; /
```

Instruções IF-THEN-ELSE

```
IF (condição) THEN
    -- relação_de_comandos
ELSIF (condição) THEN
    -- relação_de_comandos
ELSE
    -- relação_de_comandos
END IF;
```

Saída no terminal

DBMS_OUTPUT.PUT_LINE ('mensagem')

- Permite enviar mensagens a partir de um bloco PL/SQ tais como procedures e triggers
- Útil para depuração de código
- Saída no console deve ser habilitada com `SET SERVEROUTPUT ON`

-- Exemplo

```
DECLARE
    valor    NUMBER(2) := 4;
    tipo     VARCHAR2(5);
BEGIN
    IF MOD(valor,2) = 0 THEN
        tipo := 'PAR';
    ELSE
        tipo := 'IMPAR';
    END IF;
    DBMS_OUTPUT.PUT_LINE ('O número é: ' || tipo);
END; /
```

Instrução CASE

- Retorna resultado de acordo com valor da variável de comparação.

```
[variável :=]
```

```
CASE
```

```
    WHEN expressão_1 THEN declaração_1
```

```
    WHEN expressão_2 THEN declaração_2
```

```
    . . .
```

```
    ELSE declaração_n
```

```
END;
```

Instrução CASE

Exemplo:

```
DECLARE
    V_RA      ALUNO.RA%TYPE := 1;
    V_NOTA    ALUNO.NOTA%TYPE;
    V_CONCEITO VARCHAR2(12);
BEGIN
    SELECT NOTA
    INTO V_NOTA
    FROM ALUNO
    WHERE RA = V_RA;
    V_CONCEITO :=
        CASE
            WHEN V_NOTA <= 5 THEN 'REGULAR'
            WHEN V_NOTA < 7 THEN 'BOM'
            ELSE 'EXCELENTE'
        END;
    DBMS_OUTPUT.PUT_LINE ('Conceito: ' || V_CONCEITO);
END; /
```

Laços de repetição - FOR

```
FOR v_contador IN valor_inicial..valor_final  
LOOP  
    bloco_de_comandos  
END LOOP;
```

-- exemplo

```
DECLARE  
    V_AUX    NUMBER(2) := 0;  
BEGIN  
    FOR V_CONTADOR IN 1..10 LOOP  
        V_AUX := V_AUX + 1;  
        DBMS_OUTPUT.PUT_LINE (V_AUX);  
    END LOOP;  
END; /
```

Laços de repetição - WHILE

- Repete um bloco de comandos enquanto a condição que segue o comando WHILE for verdadeira.

-- Exemplo

```
DECLARE
    V_AUX    NUMBER(2)  := 0;
BEGIN
    WHILE V_AUX < 10 LOOP
        V_AUX := V_AUX + 1;
        DBMS_OUTPUT.PUT_LINE (V_AUX);
    END LOOP;
END;
/
```


Laços de repetição – EXIT

-- EXIT, interrompe a execução de um laço

```
DECLARE
```

```
    V_AUX    NUMBER(2) := 0;
```

```
BEGIN
```

```
    FOR V_CONTADOR IN 1..15 LOOP
```

```
        V_AUX := V_AUX + 1;
```

```
        DBMS_OUTPUT.PUT_LINE (V_AUX);
```

```
        EXIT WHEN V_CONTADOR = 10;
```

```
    END LOOP;
```

```
END;
```

```
/
```

Laços de repetição – LOOP

-- Executa bloco de comandos até que uma instrução de saída (EXIT) seja encontrada.

```
DECLARE
    V_AUX    NUMBER(2)  := 0;
BEGIN
    LOOP
        V_AUX := V_AUX + 1;
        DBMS_OUTPUT.PUT_LINE (V_AUX);
        IF V_AUX = 10
            THEN EXIT;
        END IF;
    END LOOP;
END;
/
```

Tratamento de exceções

- Permite tratar erros ocorridos durante a execução de um bloco PL/SQL.
- Nesses casos, o gerenciador de banco de dados aborta a execução e procura uma entrada no bloco de exceções.
- As exceções podem ser:
 - Predefinidas
 - Definidas pelo usuário

Tratamento de exceções

```
EXCEPTION
    WHEN nome_da_exceção THEN
        relação_de_comandos;
    WHEN nome_da_exceção THEN
        relação_de_comandos;
```

-- exemplo com exceções pré-definidas

```
DECLARE
    V_ID ALUNO.ID%TYPE;
    V_NOME ALUNO.NOME%TYPE;
BEGIN
    SELECT ID, NOME
    INTO V_ID, V_NOME
    FROM ALUNO
    WHERE ID=30;
    DBMS_OUTPUT.PUT_LINE(V_ID || ' - ' || V_NOME);
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE ('Não há nenhum aluno com este ID');
    WHEN TOO_MANY_ROWS THEN
        DBMS_OUTPUT.PUT_LINE ('Há mais de um aluno com este ID');
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE ('Erro desconhecido');
END;
```

Tratamento de exceções

- Exceções definidas pelo usuário devem ser declaradas e chamadas explicitamente pelo comando RAISE

```
DECLARE
    nome_da_exceção EXCEPTION;
BEGIN ...
    IF ... THEN
        RAISE nome_da_exceção;
    END IF;
    ...
EXCEPTION
    WHEN nome_da_exceção THEN
        relação_de_comandos
END;
/
```

Tratamento de exceções

-- exemplo com exceções definidas pelo usuário

```
DECLARE
    V_ID ALUNO.ID%TYPE;
    V_CONTA NUMBER(2);
    CONTA_ALUNO EXCEPTION;
BEGIN
    SELECT COUNT(ID)
    INTO V_CONTA
    FROM ALUNO;
    IF V_CONTA > 9 THEN
        RAISE CONTA_ALUNO;
    ELSE
        INSERT INTO ALUNO VALUES (20, 'SILVA');
    END IF;
EXCEPTION
    WHEN CONTA_ALUNO THEN
        DBMS_OUTPUT.PUT_LINE('Turma cheia');
END;
/
```

Tratamento de exceções

- Para gerar um erro a partir de um exceção, utiliza-se a função armazenada
`RAISE_APPLICATION_ERROR (-numero, 'mensagem')`

Ex:

```
DECLARE
    V_CONTA NUMBER(2);
    NUMERO_EXCEDIDO EXCEPTION;
BEGIN
    ...
    IF V_CONTA > 50 THEN
        RAISE NUMERO_EXCEDIDO;
    ELSE
        ...
    END IF;
EXCEPTION
    WHEN NUMERO_EXCEDIDO THEN
        RAISE_APPLICATION_ERROR(-20001, 'Maximo excedido');
END;
/
```

Procedures

- Blocos de PL/SQL que executam sem retornar valores
- Não podem ser usados em consultas SQL
- São criados seguindo a sintaxe:

```
CREATE [OR REPLACE] PROCEDURE nome_procedure
    (argumento1 modo tipo_de_dados,
     argumento2 modo tipo_de_dados,
     argumentoN modo tipo_de_dados)
IS ou AS
    variáveis locais, constantes, ...
BEGIN
    ...
END nome_procedure;
```


Procedures

- Argumentos podem ser passados de três modos
 - IN (padrão): do ambiente → procedure; não pode ser alterado
 - OUT (passagem por referência): do procedure → o ambiente
 - IN/OUT: passa valor para o procedure; este pode ser alterado dentro do procedure e retornado para o ambiente
- Execução é feita através do comando EXECUTE (ou EXEC)

```
-- exemplo
CREATE OR REPLACE PROCEDURE soma (p1 IN NUMBER, p2 IN NUMBER) IS
    total number;
BEGIN
    total := p1 + p2;
    DBMS_OUTPUT.PUT_LINE(p1 || '+' || p2 || '=' || total);
END soma;
/

SET SERVEROUTPUT ON;
EXEC soma(10, 15);
```

Procedures - exemplos

```
CREATE OR REPLACE PROCEDURE reajusta (vcateg IN produto.categoria%TYPE,  
    vpercent NUMBER) IS  
BEGIN  
    UPDATE produto  
    SET valor = valor*(1+vpercent/100)  
    WHERE categoria = vcateg;  
END aumenta_valor;  
/
```

```
CREATE OR REPLACE PROCEDURE inserecidade(vcod in cidades.codigo%TYPE,  
    Vnome IN cidades.nome%TYPE, vuf IN cidades.uf%TYPE) IS  
BEGIN  
    INSERT INTO cidades (codigo, nome, uf) VALUES (vcod, vnome, vuf);  
  
    EXCEPTION  
        WHEN DUP_VAL_ON_INDEX THEN  
            DBMS_OUTPUT.PUT_LINE('ERRO!Codigo ja cadastrado');  
END inserecidade;  
/
```